

Kick-off & *fondations.*

App Router, routing avancé
& Server Components.

STACK

Next.js 14+
React 18+

FORMAT

Cours • Pratique
Cadrage projet



Au programme de la *séance*.

10 CHAPITRES · 4 EXERCICES

01	Introduction à Next.js	p. 02	02	Projet production-ready	p. 03
03	L'App Router	p. 05	04	Layouts & pages spéciales	p. 07
05	Routes dynamiques & groups	p. 10	06	Server vs Client Components	p. 12
07	Navigation & prefetching	p. 14	08	Structure & conventions	p. 16
09	Cadrage du projet fil rouge	p. 18	10	Exercices & glossaire	p. 20

01

Un framework
fullstack *basé sur*
React.

Next.js comble les manques de React (UI seule) en ajoutant routing, rendu serveur, optimisations automatiques et API internes. C'est aujourd'hui le framework React le plus utilisé en production.

React seul, ou *Next.js* ?

React

BIBLIOTHÈQUE UI

01 Composants UI uniquement

02 Pas de routing intégré (react-router à installer)

03 Rendu client uniquement · SPA

04 Configuration manuelle (Webpack, Babel)

05 Pas d'API intégrée

06 SEO difficile, tout en client-side

Next.js

FRAMEWORK FULLSTACK

01 UI + backend, dans un même projet

02 Routing basé sur le système de fichiers

03 SSR, SSG, RSC, streaming

04 Zero-config & optimisations automatiques

05 Route Handlers · API internes

06 SEO natif grâce au rendu serveur

Pages Router, ou *App Router* ?

ASPECT	PAGES ROUTER • LEGACY	APP ROUTER • STANDARD
Dossier	/pages	/app
Composants par défaut	Client Components	Server Components
Layouts	Via _app.js • un seul global	Imbriqués par dossier
Data fetching	getServerSideProps • getStaticProps	fetch() dans Server Components
Routes API	pages/api/ ...	app/api/ ... /route.js

Dans ce cours, l'App Router • exclusivement.

STANDARD RECOMMANDÉ DEPUIS NEXT.JS 13

02

Production-*ready*
dès le
premier commit.

Un projet pro définit ses règles de qualité dès le départ · ESLint, Prettier, TypeScript strict, conventions de commit, .env.example. Les bonnes habitudes coûtent moins cher tôt que tard.

02.2 CRÉATION & STRUCTURE

Une commande, *un projet*.

TERMINAL • ZSH

CREATE-NEXT-APP

```
# Créer un nouveau projet npx create-next-app@latest mon-projet #
Réponses recommandées TypeScript? . . . . . Yes ESLint? . . . . .
. . Yes Tailwind? . . . . . Yes src/ ? . . . . . Yes
App Router? . . . . . Yes // obligatoire @/* alias? . . . . .
Yes $ cd mon-projet $ npm run dev → http://localhost:3000
```



03

Chaque dossier
est un *segment*
d'URL.

Dans l'App Router, l'arborescence du dossier /app dessine littéralement l'arborescence de l'application. Les fichiers spéciaux (page, layout, loading, error...) définissent le comportement de chaque segment.

03.1 LES FICHIERS SPÉCIAUX

Sept fichiers, *sept rôles*.

page.tsx	L'interface UI de la route · ce que voit l'utilisateur.	OBLIGATOIRE
layout.tsx	Wrapper partagé : header, sidebar, footer. Conservé entre navigations.	RACINE • OUI
loading.tsx	UI affichée pendant le chargement · Suspense automatique.	RECOMMANDÉ
error.tsx	Error Boundary React · capture les erreurs du segment.	RECOMMANDÉ
not-found.tsx	UI 404 quand la route n'existe pas.	RECOMMANDÉ
template.tsx	Comme un layout, mais re-crée à chaque navigation. Pour animations.	OPTIONNEL
route.ts	Route Handler · API interne. Étudié en S03.	OPTIONNEL

L'arborescence *est* l'app.

ARBORESCENCE / APP

```
app/
├─ page.tsx
├─ about/page.tsx
├─ blog/
│   ├─ page.tsx
│   └─ [slug]/page.tsx
└─ dashboard/
    ├─ layout.tsx
    ├─ page.tsx
    ├─ settings/page.tsx
    └─ users/
        └─ [id]/page.tsx
```

URLS GÉNÉRÉES

app/page.tsx	→ /
about/page.tsx	→ /about
blog/page.tsx	→ /blog
blog/[slug]/page.tsx	→ /blog/mon-article
dashboard/page.tsx	→ /dashboard
dashboard/settings/page.tsx	→ /dashboard/settings
users/[id]/page.tsx	→ /dashboard/users/42

Des layouts *imbriqués*, naturellement.

APP/DASHBOARD/LAYOUT.TSX

SERVER COMPONENT

```
export default function DashboardLayout({ children, }: { children:
React.ReactNode }) { return ( <div className="dashboard"> <aside>
<nav> <a href="/dashboard">Accueil</a> <a
href="/dashboard/settings">Paramètres</a> </nav> </aside> <section>
{children}</section> </div> ); }
```

LAYOUT

Conservé entre les navigations.

L'état est préservé, la sidebar ne re-render pas. Toutes les pages enfants partagent automatiquement la mise en page.

TEMPLATE

Re-crée à chaque navigation.

L'état est réinitialisé. À utiliser uniquement pour des animations d'entrée ou un reset volontaire.

PAGES D'ÉTAT

loading · error · not-found

Suspense automatique, Error Boundary, 404 personnalisée · chaque segment peut avoir les siens.

Capter l’URL, *ou la cacher.*

[slug]

Un seul segment dynamique.

```
/blog/[slug]  
→ /blog/hello
```

[... slug]

Catch-all : tous les segments suivants.

```
/docs/[ ... slug]  
→ /docs/a/b/c
```

[[... slug]]

Catch-all optionnel · matche aussi le vide.

```
/shop/[[ ... slug]]  
→ /shop · /shop/cat/id
```

(PARENTHÈSES)

Route groups · invisibles dans l’URL.

Un dossier (group) organise le code et permet de partager un layout entre des pages qui ne sont pas dans la même hiérarchie d’URL.

```
app/  
├─ (marketing)/           # layout pub + CTA  
│   ├─ layout.tsx  
│   ├─ page.tsx           → /  
│   └─ about/page.tsx    → /about  
└─ pricing/page.tsx      → /pricing  
└─ (app)/                 # layout sidebar app  
    ├─ layout.tsx  
    ├─ dashboard/page.tsx → /dashboard  
    └─ settings/page.tsx  → /settings
```



PAR DÉFAUT : TOUT EST SERVER COMPONENT.

Deux environnements, une *frontière*.

- Par défaut, tous les composants s'exécutent *uniquement sur le serveur*.
- La directive 'use client' bascule un composant côté navigateur.
- Règle d'or : descendre la frontière le plus bas possible.

Où s'exécute *le code* ?

Server Component

PAR DÉFAUT • AUCUNE DIRECTIVE

↳ **Exécution** : uniquement sur le serveur

↳ **useState** • **useEffect** : indisponibles

↳ **Event handlers** : impossibles

↳ **fetch BDD/API** : async/await natif

↳ **Secrets .env** : accessibles

↳ **Bundle JS** : 0 KB envoyé au client

↳ **SEO** : excellent • HTML rendu serveur

Client Component

DIRECTIVE 'USE CLIENT'

↳ **Exécution** : SSR puis hydratation client

↳ **useState** • **useEffect** : disponibles

↳ **Event handlers** : onClick, onChange...

↳ **fetch BDD/API** : via useEffect + loading

↳ **Secrets .env** : exposés au navigateur

↳ **Bundle JS** : code + dépendances envoyés

↳ **SEO** : bon • SSR puis hydratation

06.2 EXEMPLES CONCRETS

Deux composants, *deux mondes*.

APP/USERS/PAGE.TSX · SERVER0 KB JS

```
// S'exécute SUR LE SERVEUR async function UsersPage() { // fetch
direct, sans useEffect const res = await fetch(
'https://api.example.com/users' ); const users = await res.json();
return ( <ul> {users.map(u => <li key={u.id}>{u.name}</li> )} </ul>
); }
```

COMPONENTS/COUNTER.TSX · CLIENT'USE CLIENT'

```
'use client'; // obligatoire import { useState } from 'react'; export
default function Counter() { const [count, setCount] = useState(0);
return ( <button onClick={() => setCount(count + 1)}> Cliqué {count}
fois </button> ); }
```

Fetch BDD ou API · accès aux secrets · contenu statique serveur

■ SERVER COMPONENT

useState · useEffect · onClick · localStorage · APIs navigateur

■ CLIENT COMPONENT

Naviguer *sans recharger.*

LINK · NAVIGATION DÉCLARATIVE

PREFETCH AUTO

```
import Link from 'next/link'; <Link href="/">Accueil</Link> <Link href="/about">À propos</Link> <Link href={` /blog/${slug}`}>{title}</Link> <Link href="/dashboard" prefetch={false}> Dashboard </Link>
```

USEROUTER · NAVIGATION PROGRAMMATIQUE

'USE CLIENT'

```
'use client'; import { useRouter } from 'next/navigation'; const router = useRouter(); router.push('/dashboard'); // navigation router.replace('/dashboard'); // sans historique router.back(); // retour arrière router.refresh(); // re-fetch RSC
```

<LINK>

Navigation client + prefetch automatique au survol.

USEPATHNAME()

L'URL courante · pour des liens actifs, des breadcrumbs.

USESEARCHPARAMS()

Lit les query params : ?page=2, ?q=..., ?filter=...

Une structure *qui scale.*

```
src/
├─ app/                # Routes
│  ├─ layout.tsx
│  ├─ page.tsx
│  ├─ (marketing)/    # public
│  ├─ (app)/          # auth
│  └─ api/            # handlers
├─ components/
│  ├─ ui/             # base
│  └─ features/       # métier
├─ lib/               # db, zod
├─ hooks/
├─ services/
└─ types/
```

CONVENTIONS DE NOMMAGE

- PascalCase · Composants & types
- camelCase + use · Hooks personnalisés
- kebab-case · Dossiers d'URL
- page.tsx · layout.tsx · minuscules

```
IMPORT ALIAS @/*

// MAUVAIS · chemins relatifs fragiles
import { Button } from '../..../components/ui/Button';

// BON · alias stable
import { Button } from '@components/ui/Button';
```

Six contraintes, *un MVP*.

01 LAYOUTS IMBRIQUÉS Au moins deux zones distinctes · public + privé.	02 DATA FETCHING SERVEUR Quelles pages affichent des données ? D'où viennent-elles ?	03 SERVER ACTION Un formulaire de création ou modification.	04 ROUTE HANDLER Une API interne · webhook, endpoint public.	05 AUTH NEXT-AUTH Une feature user-specific · profil, favoris, notes.	06 OPTIMISATIONS MESURABLES Images, polices, streaming.
--	---	--	---	--	--

“

En tant qu’*utilisateur*, je veux *créer une recette*, afin de la retrouver dans *mon espace personnel*.

Un MVP livré > un produit parfait non livré.

RÈGLE D’OR • COMMENCEZ PETIT, FAITES TOURNER LE FLUX COMPLET, PUIS ENRICHISSEZ.

Quatre exercices, *un projet vivant*.

EX 01	Mise en place du projet Créer un projet avec create-next-app, activer l'App Router, initialiser un dépôt Git, premier commit « chore: initial commit », push GitHub, branche develop.	≈ 30 min
EX 02	Routing de base Créer les routes / • /about • /blog • /blog/[slug] • /dashboard • /dashboard/settings, avec des pages minimales et des données en dur.	≈ 30 min
EX 03	Layouts & pages d'état Layout racine (header, footer), layout /dashboard avec sidebar, loading.tsx global et /dashboard, error.tsx, not-found.tsx personnalisé.	≈ 30 min
EX 04	Server / Client Components Composant Counter (Client, useState) intégré dans la page / (Server). Vérifier dans les DevTools que le bundle JS ne contient que le Counter.	≈ 20 min

↳ Prochaine étape

Cadrez votre projet, *commitez.*

D'ICI LA S02

Finalisez le cadrage du projet fil rouge et un backlog priorisé en user stories.

SÉANCE 02 • AU PROGRAMME

Data fetching serveur, cache & revalidation, Server Actions.